

Monnit Utility Security Architecture & Compliance Brief

Executive Summary

The Monnit MODBUS TCP Utility is a locally hosted native application designed for monitoring and configuring industrial IoT gateways. It operates on a "**Local-First**" **data sovereignty model**, ensuring that no sensor data is transmitted to third-party cloud services unless explicitly configured via outbound plugins.

The application architecture strictly adheres to modern Electron security best practices, enforcing a hard separation between the **Main Process** (System Level) and the **Renderer Process** (User Interface) via a secure Preload Bridge.

1. Security Architecture & Isolation

The application employs a sandboxed architecture to prevent Remote Code Execution (RCE) and Cross-Site Scripting (XSS) escalation.

1.1 Process Sandboxing

- **Context Isolation (contextIsolation: true):** Enabled in main.js. The Renderer process runs in a separate JavaScript context from the Main process. Client-side scripts (client-core.js, etc.) cannot access Electron internal APIs or Node.js primitives directly.
- **Node Integration Disabled (nodeIntegration: false):** Enabled in main.js. The Renderer process is strictly prohibited from requiring Node.js modules (e.g., fs, child_process), mitigating the risk of malicious scripts accessing the underlying operating system.

1.2 Inter-Process Communication (IPC) firewall

Communication between the UI and the System Core is mediated exclusively by preload.js. This script acts as a security checkpoint.

- **Channel Whitelisting:** The application does **not** allow arbitrary IPC messages.
 - validSendChannels: A strict array defining allowed commands (e.g., start-polling, save-mqtt-config).
 - validReceiveChannels: A strict array defining allowed data streams (e.g., modbus-data, system-status-update).

- **Attack Surface Reduction:** Any IPC message not explicitly defined in these arrays is blocked and logged, preventing unauthorized access to backend functions.

2. Data Persistence & Integrity

2.1 Local Data Storage

- **Database Engine:** SQLite (better-sqlite3).
- **Location:** App User Data Directory (%APPDATA%/Monnit Utility/sensor_data.db on Windows).
- **Data Sovereignty:** Data remains on the host machine. The application does not contain telemetry or "phone-home" mechanisms to Monnit servers, other than optional iMonnit connectivity configured on the Gateway itself.
- **Encryption Status: Data is NOT encrypted at rest.** The SQLite database stores sensor readings and configuration logs in plain text on the local disk. Access controls rely on the operating system's file permission model.
- **Transmission Security: Data is NOT encrypted during transmission** between the application logic and the local SQLite file. Since the database resides on the same local filesystem as the application, network encryption (TLS/SSL) is not applicable to this internal I/O operation.

2.2 SQL Injection Mitigation

- **Prepared Statements:** All database operations in main.js utilize **Prepared Statements** (e.g., db.prepare('INSERT VALUES (?, ?)').
- **Sanitization:** Inputs are bound to parameters rather than concatenated into SQL strings, rendering the application immune to classic SQL Injection attacks via sensor names or configuration fields.

3. Network Operations & Firewall Requirements

The application acts primarily as a Local Network Client but includes optional Server capabilities via plugins.

3.1 Core Network Dependencies (Outbound)

Protocol	Port	Direction	Purpose
MODBUS TCP	502	Outbound	Polling sensor registers from Gateways.

HTTP	80	Outbound	Scraping legacy status pages (status.htm) from Gateways.
UDP	30303	Outbound	avahi-browse / mDNS for Gateway discovery scans.

3.2 Plugin Network Dependencies (Optional)

These ports are only active if the specific plugin is enabled in the configuration.

Plugin	Protocol	Port (Default)	Direction	Security Note
SNMP Trap	UDP	162	Inbound	Requires bind permission. Filters by Community String.
BACnet	UDP	47808/47809	Inbound/Outbound	Broadcasts device presence on local subnet.
MQTT	TCP	1883 / 8883	Outbound	Supports TLS (mqttts://) and Authentication.
LoRaWAN	UDP	1700	Inbound	Listens for Semtech Packet Forwarder traffic.
Webhooks	TCP	80 / 443	Outbound	Supports HTTPS. Credentials stored locally.

3.3 Update Mechanism Network Requirements (New)

The application's auto-update feature (electron-updater) requires outbound HTTPS access to download release artifacts. If operating in a strictly firewalled environment, the following allow-list rules are required to permit updates:

- **Protocol:** HTTPS (TCP)
- **Port:** 443
- **Destinations:**
 1. github.com (For checking release metadata and version tags)
 2. objects.githubusercontent.com (AWS S3 bucket where GitHub stores the actual binary installer files)

Note: If these URLs are blocked, the application will continue to function normally but will fail to retrieve updates. The user will see an "Update check failed" error in the logs.

4. Authentication & Access Control

4.1 Application Access

- **Single User:** The utility is designed as a single-user desktop application running under the privileges of the active OS user.
- **File System Access:** Restricted to the installation directory and the User Data directory.

4.2 Device Authentication

- **Gateway Read-Access protection:** The application respects the Monnit Gateway "Read-Access" status. It cannot modify configurations on a write-protected gateway without the device first being enabled via hardware.
- **Plugin Auth:** Credentials for MQTT and Webhooks are stored in the local SQLite database (settings table).

5. Dependencies & Supply Chain

- **Runtime:** Node.js (via exe wrapper).
- **Key Libraries:**
 - jsmodbus: Standard implementation of the MODBUS protocol.
 - axios: For HTTP requests.
 - better-sqlite3: High-performance synchronous DB driver.
- **Vulnerability Management:** Dependencies are locked via package-lock.json to ensure consistent and verified builds.

6. Operational Recommendations for Administrators

1. **Least Privilege:** Run the application as a standard user, not as Administrator/Root, unless binding to privileged ports (e.g., SNMP port 162) is explicitly required.
2. **Network Segmentation:** It is recommended to place Monnit Gateways and this Utility on a dedicated IoT VLAN to isolate MODBUS traffic (which is unencrypted by design) from the corporate network.
3. **Updates:** Ensure the application is updated.



4. **Physical Security:** Because data at rest is unencrypted, physical access to the host machine should be restricted to authorized personnel to prevent unauthorized extraction of the SQLite database file.